



IMPLEMENTATION PLAN / 28 SEPTEMBER 2026

Build the Listening Practice Loop

Adaptive Musical Practice System

A research-informed, eight-week route from microphone input to trustworthy, measurable practice recommendations.

Implementation principle Ship one reliable vertical slice first: capture a short exercise, analyse it, explain the evidence, assign one targeted task, and measure the next attempt.

Executive implementation decision

Recommended build

Build a browser-first application with a deliberately split audio pipeline. A lightweight YIN detector supplies immediate pitch feedback, while a server-side post-session pipeline benchmarks pYIN and the newer SwiftF0 model for the detailed result. Detected note events are aligned to the expected exercise with an error-tolerant dynamic program; constrained DTW is used only where time warping adds value. The recommendation engine remains deterministic. A language model may rewrite already-validated facts, but it is not allowed to measure, diagnose, or choose an unsupported exercise.

Primary outcome	A working evidence loop: <i>play</i> → <i>analyse</i> → <i>find a repeated pattern</i> → <i>assign a task</i> → <i>measure the retry</i> .
MVP exercise	A fixed, short, monophonic exercise with reference notes, sections, and tempo; maximum three minutes.
Live feedback	Current note, cents from target, signal confidence, and metronome-relative timing.
Post-session result	Note-level pitch and onset metrics, alignment confidence, problem sections, and a session summary.
Long-term result	A small set of evidence-backed patterns: register, tempo, transition, and section problems.
Delivery constraint	One instrument, one team, eight weeks. Every optional feature has a cut line.

The most important scope correction

Do not make “AI report generation” the critical path. The technically valuable part is the structured analysis and intervention loop. A deterministic report template is a complete fallback and should exist before any language-model integration begins.

Contents

Executive implementation decision	1
1 Implementation principles	4
1.1 Prove the uncertain parts first	4
1.2 Use two audio paths	4
1.3 Keep evidence and prose separate	4
1.4 Prefer confidence over false precision	4
1.5 Treat recordings as sensitive data	4

2	Research-informed audio strategy	4
2.1	What the current methods imply	4
2.2	Recommended signal path	5
2.2.1	Capture settings	5
2.2.2	Live path	5
2.2.3	Post-session path	6
3	Reference comparison and alignment	6
3.1	Do not rely on unconstrained DTW alone	6
3.2	Reference representation	6
3.3	Note segmentation	6
3.4	Per-note measurements	6
4	Selected technology stack	7
4.1	Version policy	7
4.2	Suggested repository structure	9
5	System architecture and contracts	9
5.1	End-to-end session lifecycle	9
5.2	Database model	10
5.3	Minimum API surface	10
6	Performance profile and adaptive practice	11
6.1	Pattern engine	11
6.2	Trend language	11
6.3	Intervention library	11
6.4	Measuring an intervention	12
7	Grounded report design	12
7.1	Build templates first	12
7.2	Optional language-model layer	12
8	Phased delivery plan	12
8.1	Phase 0 — Scope and benchmark lock (Days 1–3)	13
8.2	Phase 1 — Browser audio vertical slice (Days 4–8)	13
8.3	Phase 2 — Offline analysis and alignment (Week 2–3)	14

8.4	Phase 3 — API, jobs, and persistence (Week 4)	14
8.5	Phase 4 — Session review and history (Week 5)	15
8.6	Phase 5 — Adaptive practice loop (Week 6)	15
8.7	Phase 6 — Reports and product polish (Week 7)	16
8.8	Phase 7 — Evaluation, deployment, and handoff (Week 8)	16
9	Eight-week ownership map	16
10	Testing and evaluation	17
10.1	Audio benchmark corpus	17
10.2	Metrics	18
10.3	Test pyramid	18
11	Privacy, security, and data handling	18
12	Deployment and operations	19
12.1	Environments	19
12.2	Deployment units	19
12.3	Operational checks	19
13	Risk register and explicit cut line	20
14	Definition of done	21
15	Immediate next actions	21
	References	22

1 Implementation principles

1.1 Prove the uncertain parts first

The highest risks are not authentication or dashboard styling. They are pitch accuracy on the chosen instrument, reliable note segmentation, alignment in the presence of mistakes, and whether a detected pattern can be converted into a measurable practice task. The first three weeks therefore focus on executable technical spikes and a vertical slice.

1.2 Use two audio paths

Real-time feedback and accurate post-session analysis have different requirements. The live path must be small, predictable, and low-latency. The offline path can use more context and a stronger tracker. Trying to force one algorithm to satisfy both roles increases risk without improving the product.

1.3 Keep evidence and prose separate

Every user-facing claim should be traceable to stored metrics and an explicit rule. The system first produces structured facts; a report renderer then turns those facts into text. If a language model is used, its output must reference fact identifiers and pass schema and claim validation.

1.4 Prefer confidence over false precision

Low-quality frames, uncertain note boundaries, and ambiguous alignments must be marked as such. The application should say “not enough reliable evidence” instead of reporting a precise but unstable number. Long-term patterns require repeated observations from multiple sessions.

1.5 Treat recordings as sensitive data

Microphone capture requires explicit permission and secure-context delivery. The default data product is a set of features and note metrics, not a permanent recording. Raw audio is retained only when the user opts in or when an approved evaluation protocol requires it [2, 1].

2 Research-informed audio strategy

2.1 What the current methods imply

YIN remains a practical low-latency baseline because it is simple, efficient, and designed for monophonic fundamental-frequency estimation [3]. pYIN improves continuity by retaining multiple candidates and decoding them probabilistically, which makes it useful for post-session analysis [4]. CREPE demonstrated strong data-driven pitch tracking from raw waveforms [5]; CREPE Notes later showed that note segmentation still requires explicit post-processing of a pitch contour [7].

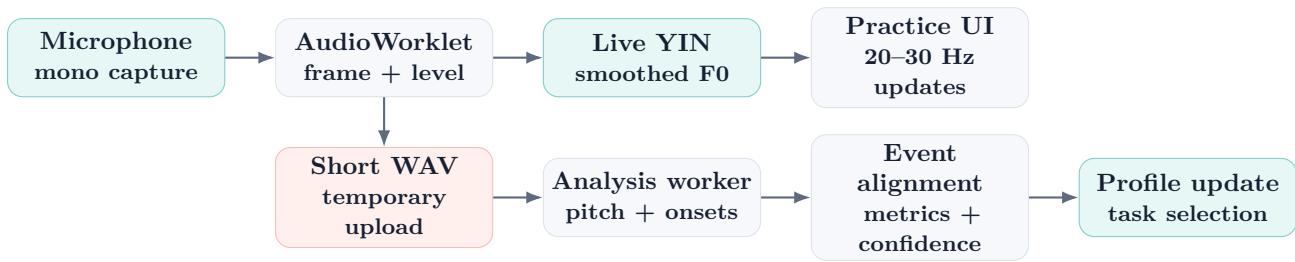
SwiftF0 is the most relevant recent candidate. Its 2025 paper and current implementation describe a lightweight monophonic tracker with confidence output, browser-capable ONNX inference, and strong benchmark results, but streaming uses approximately 176 ms of lookahead [6]. That makes it worth benchmarking for detailed analysis, while a classical detector remains the safer first live implementation.

Basic Pitch is instrument-agnostic and capable of polyphonic transcription, but the project does not need that breadth. It should be treated as a comparison or fallback transcription tool, not the default core of

a monophonic feedback loop [8].

Method	Best role	Main trade-off	Decision
YIN	Live monophonic pitch	Needs smoothing and voicing rules; susceptible to octave errors in difficult signals	Implement first
pYIN	Offline pitch baseline	More latency; not intended as the browser live loop	Benchmark and retain
SwiftF0	Strong offline or delayed pitch tracking	Emerging model; 176 ms streaming lookahead; must be tested on the chosen instrument	Preferred candidate if gate passes
CREPE	Research benchmark	Heavier than newer lightweight options	Benchmark only
Basic Pitch	Audio-to-note transcription	Solves a broader polyphonic task than required	Optional comparison

2.2 Recommended signal path



Capture settings

Request mono microphone input through `getUserMedia`. Ask the browser to disable echo cancellation, noise suppression, and automatic gain control because speech-oriented processing can distort musical pitch and dynamics. Treat these as requests, inspect the returned track settings, and show a setup warning when the browser cannot honour them. Recommend headphones so the metronome does not leak into the microphone.

Live path

The `AudioWorklet` should capture frames, compute inexpensive level information, and move reusable buffers to a worker. Avoid heavy model inference on the audio rendering thread; the Web Audio specification warns that excessive DSP work can cause glitches [1]. Start with transferable buffers. Add `SharedArrayBuffer` and cross-origin isolation only if profiling proves that copies are a bottleneck.

Use instrument-specific frequency bounds, a frame size large enough for at least two to three cycles of the lowest supported note, a 10–20 ms hop, voicing confidence, median smoothing, and octave-jump suppression. Update the interface at 20–30 Hz rather than on every audio block.

Post-session path

Keep exercises short and encode the captured mono signal as 16-bit WAV. Upload it for analysis, create a versioned feature artifact, and delete the raw file after processing unless retention is explicitly enabled. Preserve the original sample rate for onset work; resample separately to the pitch model's required rate.

3 Reference comparison and alignment

3.1 Do not rely on unconstrained DTW alone

DTW is useful for tempo variation, but a practice performance can include missed notes, extra notes, repeated notes, pauses, and restarts. A single unconstrained frame-level path can produce musically implausible matches. Research on score following commonly combines dynamic programming, feature design, and explicit error handling [9, 10].

The recommended MVP uses two levels:

1. **Event alignment.** Match detected note events to expected note events with weighted dynamic programming. Support substitution, insertion, and deletion. Cost uses pitch distance, order, expected duration, and coarse onset position.
2. **Local timing refinement.** Within high-confidence matched sections, use constrained DTW or a monotonic tempo map to refine expected versus played onset time.

This design makes mistakes visible instead of stretching the alignment until every event appears to match.

3.2 Reference representation

Store a deliberately small internal score format even if MusicXML import is added later. Each exercise requires:

- note identifier, MIDI pitch, nominal onset beat, and duration beats;
- rest events and section identifiers;
- reference tempo, supported tempo range, and count-in length;
- optional transition labels and register labels;
- transposition rule for the selected instrument;
- exercise version so historical sessions remain reproducible.

3.3 Note segmentation

Derive candidate boundaries from changes in the smoothed pitch contour, confidence troughs, energy changes, and spectral-flux onset evidence. Merge very short fragments, split only when the evidence persists, and use reference-note priors only as a soft constraint. This follows the practical lesson of recent monophonic segmentation work: accurate F0 does not automatically produce accurate note events [7].

3.4 Per-note measurements

For a matched note, exclude the attack and release margins before measuring sustained pitch. Store:

- median pitch error in cents and absolute pitch error;
- robust pitch spread, preferably median absolute deviation or interquartile range;
- voiced-frame ratio and tracker confidence;
- aligned onset error in milliseconds and beats;
- duration ratio and completion status;
- match confidence and any insertion, deletion, or substitution flag.

Never label a low-confidence match as a definite musical error.

4 Selected technology stack

4.1 Version policy

Use stable, supported releases available at project kickoff and pin them in lockfiles or container images. As of 28 September 2026, Node.js 24 is an LTS line, Python 3.14 is the latest stable Python feature series, and PostgreSQL 18 is the current stable database series; PostgreSQL 19 is still beta and Python 3.15 is still a release candidate [14, 15, 16]. Do not build the project on prereleases.

Layer	Selected stack	Why this is the practical choice
Web application	Next.js App Router, React, TypeScript, Node.js 24 LTS, pnpm	Current framework path, strong browser/API integration, typed UI, and a committed lockfile. Browser-audio components remain client components [11].
Live audio	Web Audio API, AudioWorklet, dedicated worker, Canvas	Standard browser primitives; separates the rendering thread from analysis and avoids React updates at audio rate.
API	Python 3.14, FastAPI, Pydantic	Good fit for typed JSON contracts and the Python audio ecosystem.
Persistence	PostgreSQL 18, SQLAlchemy 2, Alembic	Relational integrity for sessions, notes, patterns, and intervention trials; mature migrations and async support.
Analysis	NumPy, SciPy, librosa, SoundFile, soxr; YIN/pYIN; SwiftF0 candidate	Transparent DSP baseline plus a current lightweight neural candidate. No model training.
Jobs	RQ with Redis 8.2 extended-support line	Small operational surface, retries, timeouts, dependencies, and failed-job inspection; sufficient for an eight-week project [13].
Artifacts	S3-compatible object storage	Temporary audio and compressed contour artifacts do not belong in relational rows.
Frontend tests	Vitest and Playwright	Fast unit/component tests plus real-browser exercise flows.
Backend tests	pytest, pytest-asyncio, HTTPX	Direct tests for analysis functions, API contracts, and async endpoints.
Observability	Structured JSON logs, request IDs, job IDs, basic error tracking	Enough to trace one session from upload through profile update without introducing a large monitoring platform.

Why not a serverless-only backend?

Detailed audio analysis is CPU-bound, may use native libraries, and must survive longer than a request. Deploy the API and RQ workers as ordinary containers or processes. The Next.js frontend may use a managed web platform, but the analysis worker needs predictable runtime and memory.

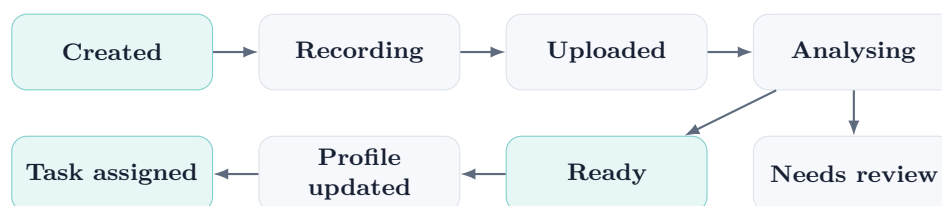
4.2 Suggested repository structure

<code>apps/web</code>	Next.js application, AudioWorklet assets, practice interface, dashboard, browser tests.
<code>services/api</code>	FastAPI routes, authentication integration, schemas, persistence, report endpoint.
<code>services/worker</code>	RQ entry point and idempotent analysis/profile jobs.
<code>packages/contracts</code>	Generated or shared API schemas and event names.
<code>analysis/core</code>	Pure audio, segmentation, alignment, metrics, and pattern functions.
<code>analysis/benchmarks</code>	Fixed audio fixtures, labelled references, benchmark scripts, and result snapshots.
<code>infra</code>	Container files, local Compose configuration, deployment notes, and migrations.
<code>docs</code>	Architecture decisions, metric definitions, data policy, and evaluation protocol.

Keep the analysis package independent of FastAPI and RQ. The same pure function should run in a notebook, unit test, benchmark, or background job.

5 System architecture and contracts

5.1 End-to-end session lifecycle



State transitions are explicit and idempotent. Re-running an analysis job for the same `session_id` + `pipeline_version` must replace or reuse the same derived artifacts rather than duplicating metrics.

5.2 Database model

Entity	Required responsibility
User / Instrument-Profile	Account ownership, instrument, transposition, supported range, preferences.
Exercise / ExerciseVersion	Stable exercise identity plus immutable notes, sections, tempo, and analysis settings.
PracticeSession	User, exercise version, tempo, timestamps, device metadata, state, and pipeline version.
AudioAsset	Temporary object key, consent/retention state, checksum, format, duration, and deletion timestamp.
SessionNoteMetric	One row per expected note: match status, pitch, timing, stability, and confidence.
SessionSummary	Versioned aggregate metrics and quality flags.
Pattern	Type, condition, effect size, evidence count, confidence, first/last observed, and status.
Intervention	Reusable rule-backed practice-task template.
InterventionTrial	Assigned pattern, dose, target metric, pre-session, follow-up session, and outcome.
ReportFact / Report	Validated facts, fact identifiers, renderer/model version, output, and validation status.

Do not store every 10 ms pitch frame as a PostgreSQL row. Store compressed contour arrays as a versioned artifact (for example, compressed NumPy data) and keep only per-note and per-session metrics relationally queryable.

5.3 Minimum API surface

- GET /exercises and GET /exercises/{id}/versions/{version}
- POST /sessions, POST /sessions/{id}/upload, and POST /sessions/{id}/complete
- GET /sessions/{id} and GET /sessions/{id}/analysis
- GET /profile/patterns and GET /profile/trends
- POST /intervention-trials and POST /intervention-trials/{id}/follow-up
- GET /reports/{session_id}

Return 202 Accepted for queued analysis and expose a job/session state endpoint. FastAPI's in-process background task is appropriate only for small work; its own documentation recommends a separate queue for heavier computation [12].

6 Performance profile and adaptive practice

6.1 Pattern engine

Start with transparent aggregations. Group reliable note observations by:

- pitch class and register band;
- directed note transition, such as F4 → G4;
- exercise section;
- tempo bucket;
- context after a rest or a large interval.

For each group, calculate evidence count, number of distinct sessions, robust median error, spread, recent-versus-baseline difference, and a confidence interval or bootstrap interval. A pattern is publishable only when it clears all of the following gates:

1. sufficient reliable notes;
2. evidence from multiple sessions;
3. an effect larger than a predefined practical threshold;
4. no strong contradiction from recent high-confidence data;
5. a matching intervention exists.

Recommended initial threshold: at least eight reliable note observations across at least three sessions. Tune this with pilot data rather than presenting it as a universal musical rule.

6.2 Trend language

Use cautious categories: *insufficient evidence*, *emerging*, *persistent*, *improving*, *stable*, and *regressing*. Compare a recent window with a baseline window and store the measured difference. Do not infer a cause from correlation with tempo or register.

6.3 Intervention library

Every intervention is a controlled object, not free-form generated advice. It defines:

- eligible pattern types and exclusions;
- exercise or section selector;
- starting tempo rule and progression rule;
- repetition count and stop condition;
- target metric and success threshold;
- follow-up timing and maximum number of retries.

Example: for a tempo-dependent onset problem, isolate the smallest affected section, start at 80–85% of the observed threshold tempo, request three reliable repetitions, then increase by 4 BPM. Record the exact task so the follow-up measurement is interpretable.

6.4 Measuring an intervention

Link a pre-intervention session, the assigned task, and a follow-up session. Compare the same section and tempo where possible. Report immediate change as a measured difference, not proof of learning. A stronger claim requires repeated successful trials on later days or a held-out exercise.

7 Grounded report design

7.1 Build templates first

Create a deterministic report from a validated fact bundle before integrating a language model. The bundle contains only facts that may appear in the report, such as:

- `fact_17`: overall pitch MAE improved by 4.2 cents from the previous comparable session;
- `fact_23`: F4–G4 has a median error of -18 cents over 11 reliable observations in four sessions;
- `fact_31`: intervention `tempo_ladder_v1` is eligible at 72 BPM.

7.2 Optional language-model layer

If time remains, send the validated bundle and an allowed recommendation identifier to a hosted language model. Require a JSON Schema response containing summary sentences, cited fact identifiers, and the chosen allowed task identifier [17]. Reject output containing an unknown number, unknown fact, unsupported comparison, medical or physical diagnosis, or an exercise outside the intervention library. Fall back to the deterministic report on any validation failure.

NIST identifies confident false content as a central generative-AI risk [18]. For this system, the practical response is architectural: the language model receives no authority to create measurements or recommendations.

8 Phased delivery plan

The phases are ordered by uncertainty and dependency. Each phase ends with a gate. A failed gate triggers the stated scope reduction before the next phase begins.

8.1 Phase 0 — Scope and benchmark lock (Days 1–3)

Goal: decide what can be measured reliably

Build

- Choose two candidate instruments or registers and define a short chromatic/interval test set.
- Capture labelled clean, quiet-room, ordinary-laptop, and moderate-noise examples.
- Benchmark live YIN and offline pYIN, SwiftF0, and optionally CREPE/Basic Pitch.
- Measure cents error, octave errors, voicing failures, runtime, and end-to-end display delay.
- Lock the primary instrument, supported range, microphone guidance, and exercise duration.

Artifacts Benchmark notebook/script, fixed audio fixtures, results table, architecture decision record, chosen instrument profile.

Gate 0

On controlled recordings in the chosen range, the selected offline tracker should reach at least 92% raw pitch accuracy within 50 cents with fewer than 2% octave errors. The live path should produce stable visual feedback with a measured median delay below 150 ms and p95 below 250 ms. These are project acceptance targets, not universal claims.

If the gate fails: narrow the range, require a closer microphone, choose the easier candidate instrument, or remove live note identity and show only coarse sharp/flat guidance.

8.2 Phase 1 — Browser audio vertical slice (Days 4–8)

Goal: practise one fixed exercise entirely in the browser

- Scaffold Next.js, TypeScript, AudioWorklet, worker, and Canvas pitch display.
- Implement permission, device selection, input-level check, headphone prompt, count-in, and metronome.
- Add live YIN, confidence gating, smoothing, and expected-note comparison.
- Record a short mono WAV and download it locally for inspection.
- Add deterministic replay mode so browser tests can feed a known audio fixture.

Deliverable A player can select the test exercise, complete a count-in, play, and see stable pitch feedback without audio glitches.

Gate 1

No sustained audio-thread underruns on the target laptop/browser; start/stop works five times consecutively; the same fixture produces repeatable live estimates; denial or loss of microphone permission fails gracefully.

8.3 Phase 2 — Offline analysis and alignment (Week 2–3)

Goal: turn one recording into defensible note-level metrics

- Implement preprocessing, chosen pitch tracker, onset evidence, note segmentation, and quality flags.
- Implement event-level sequence alignment with insertions, deletions, substitutions, and match confidence.
- Add constrained time refinement for high-confidence sections.
- Calculate note and session metrics from a versioned pipeline configuration.
- Produce an HTML/JSON diagnostic view with contour, boundaries, matches, and rejected evidence.

Deliverable A command or test function accepts WAV + exercise JSON and returns a versioned analysis result.

Gate 2

On the labelled fixture set: at least 90% note-event F1 within an agreed onset tolerance; at least 95% correct expected-note matches for performances without restarts; mistakes are represented as insertions/deletions rather than forced matches. Manually inspect every failure case in the benchmark set.

If the gate fails: require metronome-led, fixed-tempo exercises and simplify the alignment to a bounded event window. Defer arbitrary tempo drift and restarts.

8.4 Phase 3 — API, jobs, and persistence (Week 4)

Goal: make the vertical slice durable and repeatable

- Create PostgreSQL schema and Alembic migrations.
- Implement exercise, session, upload, analysis-status, and result endpoints.
- Add RQ worker with timeout, retry, idempotency, and failed-job handling.
- Store raw audio temporarily and derived contours as versioned artifacts.
- Persist per-note/session metrics and enforce user ownership on every query.
- Add request/session/job correlation identifiers to logs.

Deliverable A browser session survives refresh, completes asynchronously, and can be re-opened from history.

Gate 3

The same session cannot create duplicate metrics; a killed worker job can be retried; object and database records reconcile; unauthorised users cannot read or modify another user's session.

8.5 Phase 4 — Session review and history (Week 5)

Goal: make the analysis understandable

- Build the note timeline, pitch/timing charts, summary cards, and quality warnings.
- Add comparable-session filtering by exercise version and tempo.
- Implement initial register, transition, tempo, and section aggregations.
- Add evidence thresholds and pattern lifecycle states.
- Seed synthetic histories to test improvement, regression, and insufficient evidence.

Deliverable The user can explain why a pattern was shown and open the sessions/notes supporting it.

Gate 4

Every displayed pattern exposes its evidence count, affected condition, measured effect, and confidence state. Histories below the evidence threshold produce no definitive claim.

8.6 Phase 5 — Adaptive practice loop (Week 6)

Goal: assign and evaluate a focused task

- Create three to five intervention templates covering the initial pattern types.
- Implement deterministic eligibility and ranking rules.
- Build assigned-task and follow-up-session flows.
- Compare pre/post metrics under matched conditions and store the outcome.
- Add stop rules so the system does not repeat an ineffective task indefinitely.

Deliverable At least one end-to-end intervention can be assigned, performed, and measured.

Gate 5

Every task is connected to one detected pattern and one success metric. The app distinguishes “improved in this trial” from “problem resolved.”

8.7 Phase 6 — Reports and product polish (Week 7)

Goal: communicate results without exceeding the evidence

- Implement deterministic session and longitudinal report templates.
- Build validated fact bundles and snapshot tests.
- If core gates are already green, integrate a schema-constrained language-model renderer.
- Validate fact references, numbers, comparison direction, and task identifiers.
- Improve accessibility, empty states, permission help, responsive layout, and loading states.

Deliverable A readable report that remains correct with or without the language model.

Gate 6

In the report test corpus, 100% of numerical claims are traceable to supplied facts, unsupported outputs are rejected, and template fallback always succeeds.

8.8 Phase 7 — Evaluation, deployment, and handoff (Week 8)

Goal: finish a testable system, not a collection of demos

- Run the fixed audio benchmark and browser/device matrix.
- Conduct a small consented pilot with three to five players if approval and recruitment permit.
- Measure latency, analysis time, job reliability, API performance, and report validity.
- Deploy frontend, API, worker, PostgreSQL, Redis, and object storage.
- Add backup, retention/deletion, error recovery, and deployment runbooks.
- Freeze versions; prepare demo script, architecture diagram, evaluation results, and known limitations.

Deliverable A deployed vertical slice with reproducible evaluation evidence and a documented fallback path.

9 Eight-week ownership map

This map assumes a small team. If there is only one developer, follow the same order and treat the “Product” and “Platform” columns as alternating work blocks rather than parallel streams.

Week	Audio / analysis	Platform	Product / evidence
1	Benchmark trackers; lock range; live YIN	Repo, Next.js, browser capture	Exercise format; acceptance fixtures
2	Segmentation and on-set evidence	Session skeleton; local replay	Diagnostic analysis view
3	Event alignment; note metrics	Analysis package integration	Manual error review; metric definitions
4	Versioned pipeline job	API, PostgreSQL, Redis/RQ, artifacts	History retrieval and status UI
5	Aggregations and pattern rules	Query optimisation and ownership tests	Session review and evidence drill-down
6	Intervention metrics	Trial endpoints and persistence	Task flow and matched follow-up
7	Fact bundles and validation	Report endpoint and model adapter	Dashboard polish and template reports
8	Final benchmark and failure audit	Deployment, backup, monitoring	Pilot, documentation, presentation

10 Testing and evaluation

10.1 Audio benchmark corpus

Build a small project-owned benchmark so every algorithm change is measurable. Include:

- synthesised fundamentals and harmonic-rich notes with exact pitch;
- the selected instrument playing scales, repeated notes, intervals, rests, and legato/slurred transitions;
- at least three tempos and three input conditions;
- deliberate sharp/flat, early/late, missing, and extra-note cases;
- a manually reviewed note and onset annotation file;
- a fixed train/tuning subset and untouched final evaluation subset.

Public research datasets may supplement the corpus, but the final gate must include recordings from the actual chosen instrument, exercises, microphones, and browser capture path.

10.2 Metrics

Area	Required measures
Frame pitch	Raw pitch accuracy at 50 cents, raw chroma accuracy, voicing recall/false alarm, octave-error rate.
Note detection	Precision, recall, F1, onset error distribution, offset/duration error.
Alignment	Correct match rate, insertion/deletion accuracy, section-position error, low-confidence rate.
Live experience	Capture-to-display median/p95 latency, dropped/glitched frames, CPU usage on target device.
Session processing	Wall-clock time per minute of audio, queue wait, failure/retry rate.
Pattern engine	Precision on seeded histories, false-claim rate below evidence thresholds, trend-direction accuracy.
Intervention	Correct task eligibility, stored dose, comparable follow-up, pre/post calculation.
Report	Fact coverage, unsupported claim count, numeric mismatch count, fallback success.

10.3 Test pyramid

- **Pure unit tests:** cents conversion, smoothing, note metrics, alignment costs, pattern rules, report validation.
- **Golden audio tests:** fixed WAV + reference produces versioned expected metrics within tolerances.
- **Property tests:** time scaling preserves pitch; transposition changes pitch by the expected cents; an inserted note does not shift all later matches.
- **API integration tests:** migrations, ownership, idempotent complete calls, queued job state, retry and deletion.
- **Browser tests:** permission states, replayed audio, count-in, start/stop, analysis polling, result view.
- **Manual device matrix:** current Chrome, Firefox, and Safari on at least one ordinary laptop; mobile is exploratory only.

11 Privacy, security, and data handling

- Serve capture pages over HTTPS and request microphone access only after a clear user action.
- Show an unmistakable recording state and stop the media track immediately when practice ends.
- Default to deleting raw audio after successful feature extraction; make retention opt-in and time-bounded.
- Use random object keys, private buckets, short-lived signed access, checksums, and server-side encryption.
- Authorise every exercise/session/profile request against the authenticated user; never trust a user ID supplied by the client.

- Validate file type, decoded duration, channel count, sample rate, and size; decode in the worker, not the API process.
- Rate-limit session creation, uploads, analysis jobs, and report generation.
- Store model/provider secrets only on the server and remove audio/PII from report prompts.
- Provide account export and deletion; make derived artifacts discoverable by session ID so deletion is complete.
- If people participate in evaluation, obtain the required institutional approval and consent before collecting identifiable recordings.

12 Deployment and operations

12.1 Environments

Use local, staging, and production environments with separate databases, buckets, and queue namespaces. Local development uses containers for PostgreSQL and Redis but runs the web/API processes directly for quick iteration. Staging receives production-like migrations and one worker before every production deployment.

12.2 Deployment units

- **Web:** Next.js build with HTTPS and browser security headers.
- **API:** FastAPI container with health/readiness endpoints and bounded request size.
- **Worker:** same source revision as the API, separate process, fixed concurrency, job timeouts.
- **Data:** managed PostgreSQL 18, Redis 8.2 extended-support line, private object storage.

Tag every analysis with application revision, pipeline version, tracker/version, settings hash, and exercise version. This makes results reproducible after algorithms change.

12.3 Operational checks

The minimum dashboard or scheduled check reports API availability, queue depth, oldest queued job, failed jobs, analysis duration, object-storage errors, database connection health, and raw-audio files past their deletion deadline.

13 Risk register and explicit cut line

Risk	Response	Cut if necessary
Pitch tracker fails on the instrument	Benchmark first; constrain range and capture setup; confidence-gate output	Remove note name from live view; retain post-session pitch only
Note boundaries are unreliable	Combine pitch, confidence, energy, and reference priors; inspect fixtures	Use exercises with separated attacks and fixed rests
Alignment cascades after one mistake	Event edit alignment before time refinement; preserve insertions/deletions	Fixed tempo and bounded per-note windows
Too little history for patterns	Seed tests, require multiple sessions, show insufficient evidence	Session-level report only
Interventions are hard to evaluate	Match section/tempo and store exact dose	Demonstrate one pattern/task pair only
Background processing is unstable	Idempotent jobs, checksums, retries, failed registry, timeouts	Single worker and short recordings
LLM produces unsupported prose	Fact IDs, JSON Schema, validator, deterministic fallback	Remove LLM entirely
Schedule slips	Weekly gates and non-negotiable vertical slice	Cut social, teacher, import, mobile, and expression features

The cut line, in order

Cut language-model rewriting first, then raw-recording retention, advanced graphs, arbitrary MusicXML import, flexible restarts, and live timing feedback. Do not cut the stored note metrics, evidence thresholds, intervention trial, or follow-up measurement: those are the project.

14 Definition of done

The project is complete when all **MUST** items below are true. The **SHOULD** items strengthen the result but do not block delivery.

- MUST** A user can authenticate, choose the supported exercise, grant microphone permission, complete a count-in, and record a short take.
- MUST** The live view gives confidence-gated pitch feedback without persistent audio glitches on the target device.
- MUST** A background job produces versioned note matches, pitch metrics, timing metrics, quality flags, and a session summary.
- MUST** The result page shows both measurements and uncertainty; low-confidence evidence is not presented as fact.
- MUST** Multiple sessions produce at least one evidence-thresholded pattern type.
- MUST** The system assigns one deterministic intervention and links it to a comparable follow-up session.
- MUST** The application calculates and displays the intervention's measured outcome.
- MUST** Raw-audio retention follows the stated consent and deletion policy.
- MUST** The fixed benchmark, API tests, and critical browser flow pass from a clean deployment.
- SHOULD** Three to five users complete a small consented pilot and provide usability feedback.
- SHOULD** A schema-constrained language model renders the validated report with zero unsupported claims in the test corpus.
- LATER** Multiple instruments, polyphony, expression scoring, teacher accounts, mobile apps, and generated exercises.

The final demonstration

Show the same short problem across three moments: an initial take, the system recognising a repeated condition, and a follow-up take after the assigned exercise. The strongest demo is not a colourful tuner; it is a transparent change in evidence across the complete loop.

15 Immediate next actions

1. Select two realistic candidate instruments and a tester for each.
2. Define the supported pitch range and create a 30–45 second benchmark exercise.
3. Record the first labelled fixture set under three input conditions.
4. Implement one benchmark script for YIN, pYIN, and SwiftF0 using the same metrics.
5. Choose the instrument and tracker from measured results by the end of Day 3.
6. Freeze the MVP acceptance gates and begin the browser vertical slice.

References

References

- [1] W3C Audio Working Group. *Web Audio API 1.1*. Latest published specification, accessed 28 September 2026. <https://www.w3.org/TR/webaudio/>
- [2] W3C WebRTC Working Group. *Media Capture and Streams*. Candidate Recommendation Draft. <https://www.w3.org/TR/mediacapture-streams/>
- [3] A. de Cheveigné and H. Kawahara. “YIN, a fundamental frequency estimator for speech and music.” *Journal of the Acoustical Society of America*, 111(4), 2002. <https://doi.org/10.1121/1.1458024>
- [4] M. Mauch and S. Dixon. “pYIN: A fundamental frequency estimator using probabilistic threshold distributions.” *ICASSP*, 2014. <https://doi.org/10.1109/ICASSP.2014.6853678>
- [5] J. W. Kim, J. Salamon, P. Li, and J. P. Bello. “CREPE: A convolutional representation for pitch estimation.” *ICASSP*, 2018. <https://arxiv.org/abs/1802.06182>
- [6] L. Nieradzick. “SwiftF0: Fast and Accurate Monophonic Pitch Detection.” 2025; implementation consulted 28 September 2026. <https://arxiv.org/abs/2508.18440>
- [7] X. Riley and S. Dixon. “CREPE Notes: A new method for segmenting pitch contours into discrete notes.” *Sound and Music Computing Conference*, 2023. <https://arxiv.org/abs/2311.08884>
- [8] R. M. Bittner, J. J. Bosch, D. Rubinstein, G. Meseguer-Brocal, and S. Ewert. “A Lightweight Instrument-Agnostic Model for Polyphonic Note Transcription and Multipitch Estimation.” *ICASSP*, 2022. <https://github.com/spotify/basic-pitch>
- [9] N. Orio and D. Schwarz. “Alignment of Monophonic and Polyphonic Music to a Score.” *International Computer Music Conference*, 2001. <https://hdl.handle.net/11577/1365946>
- [10] T. Nakamura, E. Nakamura, and S. Sagayama. “Real-Time Audio-to-Score Alignment of Music Performances Containing Errors and Arbitrary Repeats and Skips.” 2015. <https://arxiv.org/abs/1512.07748>
- [11] Vercel. *Next.js App Router Documentation*. Updated March 2026. <https://nextjs.org/docs/app>
- [12] FastAPI Project. *Background Tasks*. Documentation consulted 28 September 2026. <https://fastapi.tiangolo.com/tutorial/background-tasks/>
- [13] RQ Project. *RQ Documentation*. Documentation consulted 28 September 2026. <https://python-rq.org/docs/>
- [14] OpenJS Foundation. *Node.js Releases*. Node.js 24 LTS status consulted 28 September 2026. <https://nodejs.org/en/about/previous-releases>
- [15] Python Software Foundation. *Python 3.14 Release Series*. Python 3.14 stable; Python 3.15 final scheduled after the research date. <https://www.python.org/downloads/release/python-3140/>
- [16] PostgreSQL Global Development Group. *PostgreSQL Release Notes*. PostgreSQL 18 stable and PostgreSQL 19 beta status consulted 28 September 2026. <https://www.postgresql.org/docs/release/>
- [17] JSON Schema Project. *JSON Schema Draft 2020-12*. <https://json-schema.org/specification>
- [18] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1, 2024. <https://doi.org/10.6028/NIST.AI.600-1>



Measure first. Adapt second. Explain honestly.